

OpenCV (cv2) Exam Cheatsheet

`import cv2, numpy as np, matplotlib.pyplot as plt` — OpenCV images are **BGR, uint8, shape (rows=h, cols=w, 3)**.

I/O & display

Task	Code
Read	<code>cv2.imread(p) · gray: cv2.imread(p, cv2.IMREAD_GRAYSCALE)</code> (returns None on bad path)
Save	<code>cv2.imwrite("o.png", img)</code>
Show (notebook)	<code>plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); gray: plt.imshow(g, cmap="gray"); plt.axis("off")</code>
Show (window)	<code>cv2.imshow("t", img); cv2.waitKey(0); cv2.destroyAllWindows()</code>
Size	<code>h, w = img.shape[:2] · cv2.resize(img, (w2, h2)) · fx=0.5, fy=0.5</code>
Copy	<code>img.copy()</code> (slices are views!)
ROI	<code>img[y0:y1, x0:x1]</code>
Flip / rotate 90	<code>cv2.flip(img, 1) (0 vert, 1 horiz) · cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)</code>

Colour spaces

`cv2.cvtColor(img, code)`: COLOR_BGR2GRAY, BGR2RGB, BGR2HSV (H 0-179, S/V 0-255), BGR2LAB, GRAY2BGR Channels: b, g, r = `cv2.split(img) · cv2.merge([b, g, r])`

Drawing (colour is BGR, thickness -1 = filled)

```
cv2.line(img, (x1,y1), (x2,y2), color, th)
cv2.rectangle(img, (x1,y1), (x2,y2), color, th)
cv2.circle(img, (cx,cy), r, color, th)
cv2.polyline(img, [pts_int32], True, color, th) # closed polygon
cv2.fillPoly(img, [pts], color)
cv2.putText(img, "txt", (x,y), cv2.FONT_HERSHEY_SIMPLEX, scale, color, th) # y = baseline
(tw, th), base = cv2.getTextSize("txt", font, scale, th)
```

Arithmetic, masks, blending

- Blend: `cv2.addWeighted(a,  $\alpha$ , b,  $\beta$ ,  $\gamma$ )  $\rightarrow \alpha \cdot a + \beta \cdot b + \gamma$`
- `cv2.add / subtract / absdiff(a, b) (saturation) · cv2.convertScaleAbs(img, alpha=1.3, beta=0)` (contrast/brightness)
- Mask: `cv2.bitwise_and(img, img, mask=m) · bitwise_or / bitwise_not / bitwise_xor`
- Composite: `fg = and(a, a, mask=m); bg = and(b, b, mask=not(m)); out = or(fg, bg)`
- Colour mask: `cv2.inRange(hsv, lower, upper)  $\rightarrow$  255 inside range`
- Blank canvas: `np.zeros((h, w, 3), np.uint8)`

Photometric / point transformations

Point op: output pixel depends only on the **same input pixel**, $s = T(r)$ (r = input, s = output, $L = 256$ levels). Always work in float, then `np.clip(s,0,255).astype(np.uint8)`.

Op	Formula	Code
Negative	$s = 255 - r$	<code>255 - g</code> OR <code>cv2.bitwise_not(g)</code>
Linear (contrast/brightness)	$s = \alpha \cdot r + \beta$	<code>cv2.convertScaleAbs(g, alpha=1.3, beta=20)</code>
Log	$s = c \cdot \log(1+r)$, $c = 255/\log(1+\max)$	<code>c = 255/np.log(1+g.max()); s = c*np.log(1+g.astype(np.float32))</code>
Inverse log	$s = \exp(r/c) - 1$	opposite of log: compresses brights, expands darks
Gamma (power)	$s = c \cdot r^\gamma$ (r normalised to $[0,1]$)	<code>s = 255*(g/255.0)**gamma</code> OR <code>np.power(g/255.0, gamma)*255</code>
LUT (fast, any curve)	$s = \text{table}[r]$	<code>lut = np.array([255*(i/255)**γ for i in range(256)]), np.uint8); cv2.LUT(g, lut)</code>

- Log**: expands dark values, compresses bright ones. Use on images with huge dynamic range (e.g. Fourier spectrum) or dark images. Larger $c \Rightarrow$ brighter. Result is nonlinear: $r=0 \rightarrow 0$, $r=\max \rightarrow 255$.

- **Gamma:** $\gamma < 1 \rightarrow$ brightens (expands darks, like log) · $\gamma > 1 \rightarrow$ darkens (expands brights) · $\gamma = 1$ identity. Use $\gamma < 1$ for washed-out/dark images, $\gamma > 1$ for over-exposed. Corrects display/monitor gamma (≈ 2.2).
- Log vs gamma: log has a fixed curve shape; gamma family gives a whole range of curves via γ .

Piecewise-linear (contrast stretching, thresholding, slicing)

```
# Contrast stretching: map [r1, r2] -> [s1, s2], linear in 3 segments
def stretch(g, r1, s1, r2, s2):
    g = g.astype(np.float32)
    out = np.where(g < r1, g * (s1/r1),
        np.where(g <= r2, (g-r1) * (s2-s1)/(r2-r1) + s1,
            (g-r2) * (255-s2)/(255-r2) + s2))
    return np.clip(out, 0, 255).astype(np.uint8)
# Min-max stretch (full range): cv2.normalize(g, None, 0, 255, cv2.NORM_MINMAX)
# Equivalent via np.interp: np.interp(g, [0, r1, r2, 255], [0, s1, s2, 255]).astype(np.uint8)
```

- Typical choice: $r1 = \min$, $r2 = \max$, $s1 = 0$, $s2 = 255$ (full stretch); $(r1,s1)=(r2,s2) \Rightarrow$ **thresholding** (binary output).
- **Gray-level slicing:** highlight range $[A, B]$: $s = 255$ if $A \leq r \leq B$ else r (keep background) or else 0 (binary). `np.where((g>=A)&(g<=B), 255, g)`
- **Bit-plane slicing:** $(g \gg k) \& 1$ · MSB planes carry most visual info.
- Slope $> 1 \Rightarrow$ contrast increased in that range; slope $< 1 \Rightarrow$ compressed.

Histogram

Task	Code
Compute	<code>h = cv2.calcHist([g],[0],None,[256],[0,256])</code> (shape 256×1) · or <code>np.bincount(g.ravel(), minlength=256)</code>
Plot	<code>plt.hist(g.ravel(), 256, (0,256))</code> · OR <code>plt.plot(h)</code>
Colour (per channel)	<code>for i,c in enumerate("bgr"): plt.plot(cv2.calcHist([img],[i],None,[256],[0,256]), c)</code>
Normalised (PDF)	<code>p = h / h.sum()</code> · CDF: <code>np.cumsum(p)</code>
Equalise (gray)	<code>cv2.equalizeHist(g)</code>
Equalise (colour)	convert to YCrCb/HSV, equalise Y/V only , convert back: <code>y,cr,cb = cv2.split(cv2.cvtColor(img,cv2.COLOR_BGR2YCrCb)); cv2.merge([cv2.equalizeHist(y),cr,cb])</code>
CLAHE (local, limited)	<code>cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8)).apply(g)</code>
Specification / matching	map CDF of source to CDF of target: <code>np.interp(cdf_src, cdf_ref, levels)</code> (skimage: <code>match_histograms</code>)
Mask histogram	<code>cv2.calcHist([g],[0],mask,[256],[0,256])</code>

- **Reading a histogram:** mass on left = dark · right = bright · narrow = low contrast · wide/flat = high contrast · bimodal = object+background ($\rightarrow$ Otsu).
- **Equalisation:** `s_k = round(255 * CDF(r_k))` $\rightarrow$ spreads intensities to a ~flat histogram, boosts global contrast. Weakness: amplifies noise, can wash out; **CLAHE** fixes this with per-tile equalisation + clip limit (higher clip = more contrast/noise).
- Equalisation is **not invertible** and output histogram is only approximately flat (discrete levels).
- Manual equalisation: `cdf = h.cumsum(); lut = np.round((cdf-cdf.min()/(cdf.max()-cdf.min())*255).astype(np.uint8)); out = lut[g]`

Other enhancement

Op	Code
Colormap	<code>cv2.applyColorMap(g_uint8, cv2.COLORMAP_JET)</code> (BONE, HOT, ...)
Grey-world balance	scale each channel by <code>mean_gray / mean_channel</code>

Which transform? Dark image $\rightarrow$ gamma <1 / log · washed-out/bright $\rightarrow$ gamma >1 · low contrast $\rightarrow$ contrast stretch or equalisation · uneven local contrast $\rightarrow$ CLAHE · huge dynamic range (spectrum) $\rightarrow$ log · specific intensity band $\rightarrow$ slicing · need custom curve $\rightarrow$ piecewise or LUT.

Filtering

`cv2.blur(img,(k,k))` · `cv2.GaussianBlur(img,(k,k),0)` (k odd) · `cv2.medianBlur(img,k)` (salt & pepper) · `cv2.bilateralFilter(img,9,75,75)` (edge-preserving) · `cv2.filter2D(img,-1,kernel)` · `cv2.Sobel(g, cv2.CV_64F, 1, 0)` · `cv2.Laplacian(g, cv2.CV_64F)`

Morphology (binary images)

`k = np.ones((5,5), np.uint8) — cv2.erode, cv2.dilate, cv2.morphologyEx(m, cv2.MORPH_OPEN|CLOSE|GRADIENT, k)` OPEN = erode→dilate (removes small white specks) · CLOSE = dilate→erode (fills small black holes)

Thresholding

```
_ , b = cv2.threshold(g, T, 255, cv2.THRESH_BINARY) # also _INV, TRUNC, TOZERO
t, b = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) # auto T
b = cv2.adaptiveThreshold(g, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C | ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, blockSize_odd, C)
```

Global fails with uneven light → adaptive. Bimodal histogram → Otsu.

Segmentation

- **Canny:** `cv2.Canny(g, low, high)` (low:high ≈ 1:2 or 1:3; blur first)
- **Contours:** `cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE); cv2.contourArea(c), cv2.boundingRect(c) → (x,y,w,h), cv2.drawContours(img, cnts, -1, color, th), cv2.arcLength(c, True), cv2.approxPolyDP(c, 0.02*peri, True)`
- **Connected components:** `n, labels = cv2.connectedComponents(m) · n, labels, stats, cent = cv2.connectedComponentsWithStats(m)` (stats cols: x,y,w,h,area)
- **Distance transform:** `cv2.distanceTransform(mask, cv2.DIST_L2, 5)`
- **Watershed:** `thresh → open → sure_bg = dilate → dist → sure_fg = dist > f·max → unknown = sure_bg - sure_fg → markers = cc(sure_fg)+1; markers[unknown==255]=0 → cv2.watershed(bgr, markers) → boundaries markers == -1`
- **K-Means:** `px = np.float32(img.reshape(-1,3)); _ , labels, centers = cv2.kmeans(px, K, None, (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2), 10, cv2.KMEANS_RANDOM_CENTERS); out = np.uint8(centers [labels.flatten()]).reshape(img.shape)`
- **GrabCut / floodFill** exist too: `cv2.floodFill(img, mask, seed, newColor, loDiff, upDiff)`

Geometric transforms

Transform	Matrix / function
Translate	$M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \rightarrow cv2.warpAffine(img, M, (w,h))$
Rotate+scale	$M = cv2.getRotationMatrix2D((cx,cy), angle_deg, scale)$ (+angle = counter-clockwise)
Shear x	$M = \begin{bmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \end{bmatrix}$ (widen canvas by $k_x \cdot h$)
Rigid	rotation matrix (scale 1) + $M[:,2] += (tx, ty)$
Similarity	rotation matrix with scale $\neq 1$ + translation
Affine (3 pts)	$M = cv2.getAffineTransform(src3, dst3)$ (float32) → <code>warpAffine</code>
Perspective (4 pts)	$M = cv2.getPerspectiveTransform(src4, dst4) \rightarrow cv2.warpPerspective(img, M, (w,h))$
Homography (≥ 4 , noisy)	$H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)$
Rotate w/o clipping	enlarge canvas: $nw = h \sin + w \cos , nh = h \cos + w \sin $, then $M[:,2] += (nw/2 - w/2, nh/2 - h/2)$
<code>dsize</code> is (width, height) . Extra: <code>borderValue=255, flags=cv2.INTER_LINEAR.</code>	
Affine keeps parallel lines; perspective keeps straight lines only.	

Hough

```
lines = cv2.HoughLines(edges, 1, np.pi/180, 150) # (rho, theta)
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 50, minLineLength=40, maxLineGap=10) # x1,y1,x2,y2
circ = cv2.HoughCircles(g_blurred, cv2.HOUGH_GRADIENT, dp=1.2, minDist=35, param1=100, param2=25, minRadius=20, maxRadius=40)
```

Pipeline: gray → blur → Canny → (ROI mask) → Hough. threshold/param2: higher = stricter. Circle minDist ≈ diameter.

Features (SIFT)

```
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)
good = [m for m, n in cv2.BFMatcher().knnMatch(des1, des2, k=2) if m.distance < 0.75*n.distance] # Lowe ratio
src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1,1,2)
H, inl = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
```

```

box = cv2.perspectiveTransform(corners_of_template.reshape(-1,1,2), H) # bounding box in scene
cv2.polylines(scene, [np.int32(box)], True, (0,255,0), 3)
cv2.drawMatches(img1, kp1, img2, kp2, good, None, flags=2)
cv2.drawKeypoints(img, kp, None)

```

Panorama: H maps right→left; pano = warpPerspective(right, H, (wL+wR, h)); pano[:hL,:wL] = left. Shortcut: cv2.Stitcher_create().stitch([a, b]). Also: cv2.ORB_create() (binary, use NORM_HAMMING), cv2.cornerHarris, cv2.goodFeaturesToTrack.

Video

```

cap = cv2.VideoCapture(path_or_0)
while cap.isOpened():
    ok, frame = cap.read()
    if not ok: break
    ...
    cv2.imshow("w", frame)
    if cv2.waitKey(30) & 0xFF == ord("q"): break
cap.release(); cv2.destroyAllWindows()

```

cap.get(cv2.CAP_PROP_FPS), CAP_PROP_FRAME_COUNT. Write: cv2.VideoWriter("o.mp4", cv2.VideoWriter_fourcc(*"mp4v"), fps, (w,h)). Side-by-side: np.hstack((a, b)) (same height & channels). Motion / intrusion: absdiff(bg, frame) → threshold → open → findContours → area filter → overlap with zone → alarm. Or cv2.createBackgroundSubtractorMOG2().apply(frame).

Wavelets (PyWavelets)

coeffs = pywt.wavedec(sig, "db4", level=4, mode="periodization") → [cA4, cD4, cD3, cD2, cD1] · big |detail| = sudden change/anomaly · threshold: mean + 3·std or MAD · pywt.waverec(coeffs, "db4") · 2-D: LL, (LH, HL, HH) = pywt.dwt2(img, "haar")

NumPy one-liners

img.shape / dtype / min() / max() / mean() · np.clip(x, 0, 255).astype(np.uint8) · img.astype(np.float32) · np.hstack/vstack · np.where(cond) · np.unique(markers) · img.ravel() · np.power(img/255.0, g)

Quick "which method?" guide

Situation	Use
Uneven illumination, text/documents	Adaptive threshold
Dark/bright objects on plain background	Otsu
Distinct colour	HSV inRange
Touching round objects	Distance transform + watershed (or Hough circles to count)
Smooth homogeneous organ	Region growing
Straight structures (lanes, borders)	Canny + Hough lines
Round objects	Hough circles
Recognise object at any scale/rotation	SIFT + ratio test + RANSAC
Join overlapping photos	SIFT + homography (warpPerspective)
Moving object / intrusion	Background subtraction + contours
Sudden changes in 1-D signal	Wavelet detail coefficients